KANERVA: Token-Budgeted, Validity-Aware Memory for Coding Agents, Without an LLM in the Loop

Artem Katolikov*

November 2026

Abstract

Coding agents forget everything between sessions, and the memory systems built for them mostly select context by similarity, ignore what items cost in tokens, and cannot tell whether a stored fact is still true. We present KANERVA, a local memory layer that needs no LLM of its own. It scores each candidate with a calibrated relevance model, selects under a hard token budget, and keeps an item's validity apart from its usefulness. On fifteen unseen repositories in four languages it recovers 0.775 of the evidence against 0.752 for hybrid retrieval, and on facts that changed it injects an outdated version 21.0% of the time against 99.9%. On C and C++ the lead is smaller, and it does not hold on strict recall once class methods count as facts. When a small local model answers from the selected notes, no difference between methods survives correction. A host agent's verdicts on the notes it saw were right 96.7% of the time, where an implicit signal was right 29.2%, and acting on them across chains of sessions on unseen repositories has shown no measurable gain or harm. On plain-language questions the largest loss is silence: the needed note is often among the candidates, and the gate and the weight floor keep it out.

1 Introduction

A coding agent starts each session knowing nothing it learned in the last one. Native memory is a set of files loaded whole: Claude Code loads its `CLAUDE.md` files and the first 200 lines or 25 KB of a memory index, with no ranking by the task at hand [3]. Memory products add retrieval, but they select by similarity top-k or leave the choice to the agent, and they spend LLM calls on every write. The published evidence does not show that this is good enough. On LoCoMo, Mem0's own table has a full-context baseline ahead of every memory system tested [12]. Retrieval method moves accuracy far more than write strategy, and raw chunks with no LLM calls match fact extraction [68]. On coding tasks, unfiltered experience gives little or negative benefit [74], and eleven of twelve pairings of solvers with memory systems fail to beat memory-off [16].

These findings point at the read path, and at three defects in it. Selection ignores what an item costs in tokens, it ignores whether an item is still true, which is the common failure in a codebase, and it has no way to learn which notes actually help in a given project. We treat memory as budgeted selection, and we make four contributions.

1. A statement of the problem in which the surveyed systems can be compared, with elementary worst cases for what each gives up (Section 3).

2. A local method that needs no LLM of its own: a calibrated relevance model, a hard token budget, and validity kept apart from usefulness (Section 4).

3. A way to learn per project and across projects from the host agent's verdicts, a test showing that those verdicts can be trusted, and a test of acting on them, which shows no measurable gain or harm yet (Sections 4.4 and 6.10).

4. RepoMem, a coding-memory benchmark built from real repository histories, in which facts change for real and labels come from the code or its commits. We evaluate on held-out repositories in six languages and on BEAM conversations, against ten baselines.

Scope of the evidence. Most results here are *retrieval-level*: evidence recall under a token budget, tokens injected and stale injection. Recall under a budget is necessary for good outcomes but does not measure them. Two tests go further: a small local model answers from the selected notes (Section 6.11), and a real coding agent works through chains of sessions (Section 6.10). In the first, recall predicted answers poorly, and in the second, learning changed little. A controlled study found that context strategy moved correctness on coding tasks by no more than 10 to 15 points [28], so large end-to-end gains should not be assumed.

*Code and data pointers: `https://github.com/katolikov/kanerva`. DOI: 10.5281/zenodo.23849910. ORCID: 0009-0007-1585-8092.

2 Related work

Memory systems. Memory layers include OS-style paging by the agent [43], LLM-curated fact stores [12], temporal knowledge graphs with bi-temporal edge invalidation [49], linked notes [66], graph retrieval [15, 20, 21], and recursive summary trees [51]. Generative Agents scores memories by recency, importance and relevance [45]. Experience memories such as Reflexion, Voyager, ACE and ReasoningBank keep lessons or skills [42, 52, 62, 70]. All of these use a write path with LLM calls, a read path with a fixed k or an agent decision, or both. Where they handle validity at all, an LLM rewrites notes, or, in Zep/Graphiti, invalidates an edge, which is binary and depends on the extractor producing that edge.

Budgeted selection. Token-budgeted submodular selection for context is an active preprint line. PACMS applies facility location under a knapsack as a pluggable engine and lists learned relevance as future work [19]. AdaGReS scores relevance against redundancy [46], and PC-SubMax compresses prompts with a regularized objective [73]. The theory rests on Nemhauser et al. [41], on budgeted maximum coverage [29], on partial enumeration [17, 56], and on cost-benefit greedy with lazy evaluation [30, 34, 39]. MMR [8] and DPPs [31] target diversity without a budget in tokens.

Learning. Combinatorial and linear semi-bandits give regret with approximate oracles [9, 10, 33, 63], and linear submodular bandits extend this to diversified retrieval and to knapsacks [57, 67, 69], under the assumption that an item's value is a stationary function of its features. In agent memory, DeMem proves regret bounds for a K-state memory partition [75] and MemRL shows Q-value convergence [71]; RSCB-MC and MERA use bandits to choose agent actions in coding [24, 25]. None of these treats token-budgeted selection of items whose validity changes. Self-scored episodes are known to inflate rewards [5], which is a risk for any signal that comes from the agent, so Section 6.10 measures ours against labels.

Validity and forgetting. ACT-R base-level activation models the power law of forgetting [1, 2], and it has been applied to LLM memory [22]. Staleness is a measured failure: with a budget of two records, models acted on stale records in most episodes of one controlled study [40], and conflict benchmarks exist for chat [59]. Consolidation follows complementary learning systems [32, 38]. Rate–distortion views of compaction appear in [13, 53, 75]. Colaco and Lahjouji argue that repeated irreversible summarization degrades end-task error faster than retrieval-backed memory, which agrees with the context collapse reported for ACE [70], and motivates keeping sources.

Compression and parametric memory. Text compression such as LLMLingua [26, 27, 44] works with closed APIs and is complementary to selection. Gist tokens, AutoCompressors, ICAE, and KV-cache eviction [11, 18, 36, 47, 65, 72] and test-time weight updates [6, 7, 54] need access to model internals that closed APIs do not expose.

Benchmarks. LoCoMo [37], LongMemEval [64], MemoryAgentBench [23] and BEAM [60] cover conversational memory. SWE-ContextBench [74] and VibeMemBench [16] cover coding experience, MemoryCode covers multi-session coding instructions on synthetic data [48], and ContextBench covers retrieval within a task [35]. None of these, as far as we found, tests repository facts that change between sessions over real history.

3 Problem

Memory. At time t the store is a finite set M_t. An item is a tuple $m = (\text{text}, e, \tau^{\text{c}}, \tau^{\text{v}}, \pi, c, k, P)$: content, unit embedding, creation time, the last time the item was confirmed, provenance, token cost $c(m) \in \mathbb{N}_{>0}$, an optional *slot key* k that names the fact the item states, and links P to the items it was consolidated from. Items that share a key form a version chain.

Validity and usefulness. For a query q, let $\rho_t(m) = \Pr[m \text{ still holds at } t]$ and $\mu_q(m) = \Pr[m \text{ helps with } q \mid m \text{ holds}]$. The *effective weight* of an item is $w_q(m) = \rho_t(m)\,\mu_q(m)$. The two factors move for different reasons: ρ changes with time and with events in the world, μ with the query. A similarity score can express neither of them on its own, and Section 5 shows what is lost when they are merged.

Selection. Injecting $S \subseteq M_t$ has utility $U(S \mid q)$, and the context budget is a hard constraint:

$$\max_{S \subseteq M_t} \ \mathbb{E}\left[U(S \mid q_t)\right] \quad \text{s.t.} \quad c(S) := \sum_{m \in S} c(m) \le B. \tag{1}$$

When w is a probability of being needed, the expected number of needed items in S is additive,

$$F(S) = \sum_{m \in S} w_q(m), \tag{2}$$

and (1) becomes a 0–1 knapsack. This is the objective the system optimises, over candidates and with estimated weights. A richer objective could reward covering several facets of a query, or penalise near-duplicates, and would be submodular; Section 5 says what would change.

A memory policy. A policy decides what to write, what to read, what to consolidate and what to forget. Reading costs tokens at every step, writing costs LLM calls or latency, and the store costs retrieval time and noise. This paper is about the read path and about how it adapts to feedback. Model weights never change.

3.1 Existing methods in these terms

Table 1 places the surveyed systems in (1). Nearly all of them read with an additive score under a limit on the number of items, so they differ mostly in what they store.

Parametric methods (test-time training [54, 55], memory layers [7], Titans [6], gist tokens and KV-cache eviction [36, 47, 72]) keep memory in weights or attention state. They need write access to weights or to the cache, which a closed API does not expose, so they fall outside this framing.

3.2 Where these methods lose

Three elementary worst cases show the design defects a better read path has to remove. Write OPT for the optimum of (1).

Proposition 1 (Cost blindness)**.** *Any rule that fills the budget in the order of a score independent of c (top-k fill, MMR, reciprocal rank fusion, reranking) has worst-case ratio at most $1/\lfloor B/c_{\min} \rfloor$ against* OPT*, even for additive U.*

Proposition 2 (Recency as proxy)**.** *A relevant item m with $c(m) \le B$, followed by later items of total cost above B, each no larger than m, is never injected by recency-only selection, however relevant it is, whereas a rule that scores by relevance injects it whenever it outranks those items.*

Proposition 3 (Time-invariant scoring)**.** *Let two versions of a fact have equal similarity to the query, and let their timestamps be assigned uniformly. A selector whose score does not depend on timestamps ranks the outdated version first with probability* $1/2$*.*

Every k-limited row of Table 1 is cost-blind, and every row with no validity model or a recency proxy falls under Proposition 2 or 3. Zep and Graphiti avoid Proposition 3 only for facts where the extractor produced the invalidating edge. Additive selection, ours included, can also spend its budget on near-duplicates, which a coverage objective would avoid in the worst case; on our data that case made no measurable difference.

4 Method

Kanerva is built on SQLite, FTS5 and NumPy, and no component calls an LLM. A host agent, when present, writes notes and judges them through tools. The token cost of memory is what the hooks inject, at session start, per prompt and per file, plus the lists a write returns and one short agent turn at the end of a session with real work.

4.1 Write path

A write redacts secrets, drops exact duplicates and stores the note with its full-text entry and embedding. A write with a key closes the live note with that key, which stays as history. Most notes have no key, and only an agent can tell that a new note replaces an older one, so `memory_remember` returns up to five similar notes, taken in turn from word matches and close vectors. A hint tells the agent to retire a note that only held the old value, or to edit out the outdated part of a longer one. The store lists and the agent decides.

4.2 Read path

Candidates are the top 200 lexical and dense hits and, with vectors, the notes written next to the 50 best hits in the same session, since a conversation often splits one fact across adjacent turns.

Relevance. Each candidate gets a probability of being needed from gradient-boosted trees over 29 features: retrieval ranks and scores, the note's length and content, query–note overlap, role and speaker, neighbouring turns, recency, and ACT-R activation [2]. The trees are fitted on LongMemEval-S, LoCoMo, BEAM-1M and 34 RepoMem repositories, with conversations and code at equal weight, and they run on NumPy in tens of milliseconds, so a user trains nothing. Separate models cover the lexical, static-embedding, dense and reranker configurations.

Selection. The weight of a note combines that probability with validity and a learned lift, and selection maximises the sum of weights (2) under the budget (Algorithm 1). Greedy(w, B, ν) goes through candidates in order of $w_m/c(m)^\nu$ and keeps each one that still fits. Length enters as a feature and as the cost, so the model can learn that short turns are rarely needed and density does not favour short junk. Coverage and facility-location terms had no measurable effect on development data, so they are off.

Query gate. Selection alone fills the budget with the best of a bad lot, so the plugins first ask whether the store can answer the prompt at all. A logistic model over six signals of the result list (four without vectors), fitted to tell a store's own questions from questions drawn from other stores, gives no notes to a prompt below its threshold. Above it, the plugins drop selected

Table 1: Surveyed methods. "Score" is the w used by the read policy, and k denotes a limit on the number of items.

Method	Write	Score w	Constraint	Validity ρ	Learns
Top-k RAG, hybrid, rerank [14, 50]	chunks	BM25/cosine/RRF/CE	k	none	no
MMR [8]	chunks	greedy rule, λ trade-off	k	none	no
PACMS, AdaGReS [19, 46]	chunks	facility location; relevance−redundancy	$c(S) \le B$	none	no
MemGPT/Letta [43]	agent tool calls	LLM decides paging	block sizes	none	no
Mem0 [12]	LLM add/update/delete	similarity	k	LLM rewrite	no
Zep/Graphiti [49]	LLM graph extraction	hybrid + rerank	k	$\rho \in \{0, 1\}$ by edge invalidation	no
A-MEM [66]	LLM notes and links	similarity + links	k	LLM note evolution	no
HippoRAG 2 [21]	OpenIE triples	personalized PageRank	k	none	no
RAPTOR [51]	recursive summaries	similarity on all levels	k or tokens	none	no
Generative Agents [45]	observations, reflections	recency+importance+relevance	k	recency proxy	no
ACE, ReasoningBank [42, 70]	LLM-curated lessons	similarity or whole playbook	k	none	via text
Claude Code files [3]	model-decided notes	constant (query-independent)	200 lines/25 KB	none	no
Full history	everything	constant	window size	none	no
LLMLingua family [26, 44]	—	—	compresses $c(m)$	none	no

Algorithm 1 RETRIEVE(q, B, t)

$C \leftarrow$ lexical hits $\cup$ dense hits $\cup$ their same-session neighbours
Remove superseded notes and those with a newer near-twin, unless q asks for history
for $m \in C$ **do**
    $p_m \leftarrow$ relevance model on features of (m, q)
    $\rho_m \leftarrow$ class survival at age $t - \tau^{\mathrm{v}}(m)$, discounted by newer similar facts
    $w_m \leftarrow \rho_m \min(1, \ell_m p_m)$ ▷ ℓ_m: learned lift
end for
$\mathcal{S} \leftarrow \{\textsc{Greedy}(w, B, \nu) : \nu \in \{1, \frac{1}{2}\}\} \cup \{\arg\max_{c(m) \le B} w_m\}$
return $\arg\max_{S \in \mathcal{S}} F(S)$

notes with weight below 0.02. Session start is not gated, because its query is generic.

File notes. The file an agent opens says more about a change than the prompt does. When a session first touches a file, the hook returns live notes that name it, within 150 tokens per file and eight times that per session, leaving out what the prompt brought and files that many notes name. This is a plain lookup with no relevance model.

4.3 Validity

Keyed facts have exact version chains: a superseded note leaves the candidates unless the query asks for history. Text without keys falls to the *near-twin rule*: a newer fact nearly identical to an older one removes it. Twins are compared by sentence vectors and by hashed words, because a sentence model hears little difference between `FOO = 3` and `FOO = 4`, and stores without vectors use words alone. To keep the rule from firing on neighbouring facts, a twin must cover the query at least as well as the older note and hold as many of its words whole, and a note that says less by more than one word is not a newer version. Only facts replace each other, since two similar events are still two events. Natural-language updates rarely look like twins, so they are left to the agent, through the write path.

Live notes also age, by a Gamma–Poisson hazard estimated per class of fact from the store's own version history. Section 6.5 measures what this adds.

4.4 Learning from the agent

The relevance model ships fitted and cannot know how notes behave in one project. Learning rests on a *verdict*: at the end of a session with real work, where the plugin already asks the agent for a short note, it also asks which notes changed what the agent did or saved it work, and which turned out wrong. A session with no verdict teaches nothing, since silence is not evidence. Verdicts are tied to the project, and a note counts once for each injection that showed it, however often it is named.

Lift. Let h_m count the times a note helped, and let e_m sum the model's probabilities over its injections, plus one each time it misled. A rate b over all notes removes the difference in scale between an agent's marks and the model's probabilities. Evidence on one note is scarce, so it is pooled. Notes fall into groups (test commands, build commands, other keyed facts, edit trails, facts, events), and each level starts from the one above, from a group across projects to a group in project s to the note:

$$\ell_g = \frac{h_g + \kappa_g}{b\, e_g + \kappa_g}, \qquad \ell_{s,g} = \frac{h_{s,g} + \kappa_g\, \ell_g}{b\, e_{s,g} + \kappa_g},$$
$$\ell_m = \mathrm{clip}_{[\frac{1}{4}, 4]} \frac{h_m + \kappa_m\, \ell_{s,g}}{b\, e_m + \kappa_m},$$

with $\kappa_g = 10$ and $\kappa_m = 1$. With no evidence every level is 1. A note the agent found by search or beside a file has no injection on record, so it adds to its own h_m alone. A note that once helped says little about a query unlike the ones it helped on, so a rise applies in proportion to how close the query is to those queries, by cosine or by shared words, while a fall applies everywhere.

Shadow check. Lift changes nothing until it has shown it ranks better. Before a verdict updates any record, it scores the model's ordering and the learned one on pairs of a note that helped and one that did not. Lift starts once the learned ordering wins two percent more of at least 40 pairs, a verdict adds at most ten pairs, and it stops if the lead goes. A project's own record decides once it has enough pairs, and until then the record across projects.

Gate and floor. A note the gate or the floor stops is never shown, so waiting yields no evidence about it. So probes start from the first prompt: a quarter of prompts, and a tenth once a project has eight judged sessions, get a list of at most 200 tokens even if they fall up to three units below the gate, with notes down to a tenth of the floor. A setting moves only if it beats the shipped one by four sessions of worth, where a prompt is worth what it helped minus $\frac{1}{4}$ and a note minus $\frac{1}{20}$, and a prompt only a probe could show stands for up to four prompts. Probing stops in a project where verdicts do not come. The relevance model's weights never change.

Default. The plugins ship with `KANERVA_LEARN=auto`. Acting on verdicts has shown no measurable gain or harm (Section 6.10), and we keep it on because a project's gate can move no other way. `shadow` records without acting, and `0` stops asking.

4.5 Optional modules

Three modules are off by default. *Consolidation* summarises clusters of near-duplicates extractively and moves the sources to an archive that Expand restores from, following evidence that irreversible rewriting erodes detail [13, 70]. *Forgetting* archives notes idle for τ days or beyond a token capacity. *Excerpts* let a long note enter selection as the run of sentences that best covers the query, which adds a constraint that Proposition 4 does not cover.

5 Guarantees

The results below cover selection and the optional modules. The gate, the weight floor, the near-twin rule and the learned lift are heuristics with no guarantee: the floor drops items after selection, so the bound of this section applies to the set selected and not to the set injected. Section 6 measures them. Proofs are in Appendix A.

5.1 Selection

Proposition 4 (Selection guarantee)**.** *Let F be the additive objective* (2) *over the candidates with $w \geq 0$, let every candidate satisfy $c(m) \leq B$, and let* OPT *be the best value of F under the budget. The better of density greedy and the best single item has $F(S) \geq \frac{1}{2}\mathrm{OPT}$.*

An exact knapsack solver would reach OPT, and we keep the simpler rule because on development data the two are level (Section 6.2). The library can add a term for covering query facets and a facility-location term against near-duplicates. The objective is then monotone submodular and the classical cost-benefit argument gives $\frac{1}{2}(1 - \frac{1}{e}) \approx 0.316$ [30, 34], with better constants at higher cost [17, 56, 58]. Both terms are off by default, so that bound plays no part in the results below.

5.2 Selection with estimated probabilities

The model only estimates the weights, which raises the question of how estimation error reaches the selection.

Proposition 5 (From probability error to selection loss)**.** *Let p_m be the true weight of candidate m and $\hat{p}_m \geq 0$ an estimate, and write $e(S) = \sum_{m \in S} |p_m - \hat{p}_m|$. Let $S^\star$ maximise $\sum_{m \in S} p_m$ under the budget. If $\hat{S}$ is feasible and $\sum_{m \in \hat{S}} \hat{p}_m \geq \alpha \max_{c(S) \leq B} \sum_{m \in S} \hat{p}_m$, then*

$$\sum_{m \in \hat{S}} p_m \;\geq\; \alpha \sum_{m \in S^\star} p_m - \alpha\, e(S^\star) - e(\hat{S}).$$

Proposition 4 gives $\alpha = \frac{1}{2}$ for the shipped rule. An exact knapsack has $\alpha = 1$, and then the loss is at most $e(S^\star \triangle \hat{S})$.

Only the error on selected and optimal items matters, and not the error over the whole pool, which is why calibration near the top of the ranking counts for more than a good fit on the long tail of candidates. In the shipped system $\hat{p}$ is the weight w of Algorithm 1.

5.3 Why validity needs its own signal

Proposition 6 (Validity-blind learners)**.** *For any $n \geq 2$ there is an instance in which n items per round have identical features, exactly one of them is valid, and B fits one item. A learner that observes neither age nor version information earns $1/n$ per round in expectation against an optimum of 1, and its regret is $(1 - \frac{1}{n})T$ at every horizon. A learner that sees the version chain has regret 0 on the same instance.*

No amount of feedback helps here, because feedback carries no information about which copy is current. This is the formal reason to keep version chains apart from usefulness.

5.4 Optional modules

Consolidation and forgetting are opt-in. The two results below say what they guarantee, and Section 6.13 measures what they cost.

Let X be a cluster of sentences with weights p ($\sum_x p_x = 1$), costs c and cosine similarities $s \in [0,1]$ with $s(x,x) = 1$, and let $\Phi(Z) = \sum_x p_x \max_{z \in Z} s(x,z)$ and $D(Z) = 1 - \Phi(Z)$ be the distortion of a summary $Z \subseteq X$.

Theorem 1 (Rate–distortion for extractive consolidation)**.** *Let Z_ℓ be the first ℓ picks of density greedy on Φ, before any pick is skipped for lack of room, with rate $R_\ell = c(Z_\ell)$. For every summary $Z^\star$ of rate $R^\star$,*

$$D(Z_\ell) \le e^{-R_\ell/R^\star} + \big(1 - e^{-R_\ell/R^\star}\big) D(Z^\star).$$

Distortion therefore decays exponentially in rate against any reference summary. The bound averages over p and says nothing about a rare needle in the cluster, which is why consolidation here keeps the sources: if a summary is retrieved, EXPAND recovers any source at the cost of its tokens.

Theorem 2 (Store stability)**.** *Let items arrive as a Poisson process of rate a, and let each be accessed by an independent Poisson process of rate r, drawn i.i.d. from a distribution with $\mathbb{E}[(e^{r\tau} - 1)/r] < \infty$. An item is archived after an idle gap of τ. Then an item's active lifetime has mean $(e^{r\tau} - 1)/r$ given r, and the number of active items converges in distribution to* $\mathrm{Poisson}\big(a\,\mathbb{E}[(e^{r\tau} - 1)/r]\big)$.

Without forgetting the store grows linearly in t, and with an idle rule it stays bounded in distribution, biased exponentially toward frequently used items. The theorem assumes independent access processes, whereas real access is bursty.

6 Experiments

6.1 Setup

Data. *LongMemEval-S* [64] has 500 questions, each with its own store of about 490 turns. *LoCoMo* [37] has 10 long conversations. *BEAM* [60] has long conversations of about 2,000 turns, of which 35 from the 1M set serve for development and 984 questions in 55 conversations from the 100K and 500K sets for evaluation. *RepoMem*, which we introduce here, turns real repositories into memory streams, at 14 snapshots over six years. Each interval is a session in which a simulated agent records commit notes and facts observed in changed files. Facts (constants, signatures, symbol locations, dependency constraints) come from static analysis, so a fact is stale exactly when the same key held another value at an earlier snapshot. Fact queries ask for the current value, and task queries use a real later commit subject, with earlier commits on the same files as gold. RepoMem runs with slot keys, as a host agent would supply them, and without.

Test protocol. A held-out test is a set of repositories that no fit, threshold or design choice has touched. We froze the method, ran the test once with a single script, and treated the set as spent afterwards: it can then serve for development, or as a check on a later change, but never again as a test. Models were later refitted on spent repositories, and each new test used new ones, so every test below scores repositories its model had not seen. Benchmarks have no verdicts, so learning plays no part in them. Unless noted, recall and gate results measure release 0.5.0, file notes 0.5.2 and the chains 0.6.1. Later releases changed the read path only through the near-twin rule, file notes and learning, and Section 6.5 gives the effect of the rule on the main test. Appendix B lists the earlier tests.

Baselines. Recency, BM25 (FTS5), dense retrieval (all-MiniLM-L6-v2 [61]), hybrid RRF [14], hybrid followed by a cross-encoder (ms-marco-MiniLM-L-6-v2) on the top 50, MMR [8], Generative Agents scoring [45] with constant importance, facility location under a knapsack as in PACMS [19], hybrid with superseded versions removed, in the spirit of [49], and hybrid keeping only the newest of near-twins. All methods share candidate generators, token counts (`cl100k_base`) and a 14-token per-item overhead, and every ranking is filled greedily to the budget, which already makes the baselines budget-aware. Systems that need an LLM on write were not run.

Statistics. The independent unit is a conversation on BEAM and a repository on RepoMem, since queries within a conversation are dependent and seeds repeat RepoMem queries. We give bootstrap intervals over units and p-values from two-sided sign-flip permutation tests on unit means, with Holm correction across budgets within a comparison. With n units the smallest possible two-sided p is 2^{1-n}, so tests on six repositories cannot pass correction across budgets, and there we read the interval and the count of units on each side. An interval that contains zero on few units is weak evidence that two methods are equal.

6.2 Development findings

Three findings from development data shaped the design, on a fit that came before the current one. The relevance model is close to calibrated: over ten equal-count bins the expected calibration error is 0.0005 on LongMemEval-S, 0.0003 on LoCoMo, 0.0016 on RepoMem and 0.0017 on BEAM-1M, against base rates

near one percent, though the highest-scored one percent of BEAM-1M candidates are predicted at 0.349 and observed at 0.287. The knapsack solver hardly matters: mean recall is 0.457 for density greedy against 0.454 for an exact knapsack on BEAM-1M. Cost does matter on BEAM-1M, where ranking by probability alone falls to 0.431, but not on RepoMem, at 0.881 against 0.881. The candidate pool matters where a question has many evidence turns: on BEAM-1M the top 100 from each retriever reach 0.665 of the evidence, and adding the neighbours of the 50 best hits reaches 0.731.

6.3 Code repositories

The main test measures release 0.5.0 on fifteen repositories, with 2,148 queries over three seeded streams each: scrapy, celery, httpie, sanic, boto3 and channels in Python, vue core, TanStack query and immer in TypeScript, cobra, echo and bubbletea in Go, and bat, hyper and fd in Rust. The fact track reads Python source, so it exists only for the first six, and the others have task queries alone. Tables 2 and 3 give the results, and Figure 1 shows recall by budget.

Without keys and averaged over budgets, KANERVA recovers 0.775 of the evidence against 0.752 for hybrid and 0.749 for the reranker. Its lead over hybrid is +0.024 (interval [+0.014, +0.033], 15/15 repositories in its favour, p <0.001), it holds at every budget, and the lead over the reranker is +0.026 (interval [+0.018, +0.037]). With keys the difference to hybrid is +0.025 (interval [+0.016, +0.034]), so the lead does not depend on keys. The gain is on tasks: without keys at 1024 tokens task recall is 0.569 against 0.527 for hybrid, while fact recall is 0.993 against 0.998, a small loss.

The lexical model, which is what a store runs before the embedding model arrives, stays close to BM25: 0.739 against 0.727 over budgets, with 9/15 repositories in its favour. Nearly all of that difference comes from the smallest budgets, and from 512 tokens up the two are level, at +0.003 at 512 tokens and +0.003 at 4096.

Table 4 splits the result by language. KANERVA leads hybrid in each: 0.870 against 0.844 on Python, 0.545 against 0.529 on TypeScript, 0.523 against 0.509 on Go and 0.441 against 0.425 on Rust. The last three rest on three repositories each and the task track alone. At small budgets the lead fails on Go at 256 tokens and on Rust at 128 and 256, where at 128 it has 0.158 against 0.159. Outside Python the lexical model only ties BM25.

6.4 C and C++

The main test holds no C or C++, a large share of what coding agents work on. Facts here are read by regular expressions in place of `ast`: literal macros and constants, public function signatures, and where each function and type is defined. Four repositories (tmux, libuv, nlohmann/json, googletest) served for development, and Table 5 gives three held-out tests of six repositories each. The relevance model has seen neither language, and refitting it on them gained nothing out of fold (+0.002 with vectors, +0.005 lexically).

On the first two tests KANERVA leads hybrid and the reranker, with the gain on tasks. The twin rule makes the largest difference on these languages: comparing twins by words as well as vectors takes stale injection on the second test from 42.9% to 16.0%, because a sentence model misses most changed constants. A real agent on those repositories also fixed 12 of 12 changed facts.

The third test also reads templates, overloads and methods under their class, and there the lead over hybrid does not hold on strict recall (Table 6). On C KANERVA leads, at 0.939 against 0.914. On C++ it trails, at 0.816 against 0.827 for hybrid, although it leads on tasks. Most of the loss is a labelling effect. Of 3,564 fact queries, 84 lose a note at 1024 tokens that hybrid keeps, and 74 of those are location notes, most of which gave way to a newer signature note naming the same file. The value itself reached the context on 0.992 of the lost queries, and overall on 0.9992 of queries against 0.9994 for hybrid. With keys, where the rule does not apply, the lead is back at +0.023 on 6/6 repositories.

6.5 Stale facts

Table 7 covers queries whose fact changed, at 1024 tokens, on the main test. With keys, every method that follows version chains injects no stale version. Without keys, hybrid and the reranker inject one 99.9% of the time and KANERVA 34.9%. The plain rule of keeping the newest near-twin reaches 36.5%, so the gain comes from that rule and not from a probabilistic model of validity. The error, a stale version listed above or instead of the current one, stays near zero at 0.003 against 0.722 for hybrid.

The current twin rule also applies to the main test's repositories, which we checked after the fact. Recall moves by -0.001 with vectors and stale injection falls from 34.9% to 21.0%, while on BEAM nothing moves. In lexical stores recall moves by +0.002 and stale injection rises from 26.6% to 29.0%. Most of the small loss in strict fact recall is the labelling effect above.

The labels on the main test have two known flaws. Of the 1,772 changed facts, 287 (16.2%) only moved to another file and kept their value, so injecting the older note does no harm, and 7 dependency queries read console-script entries as version constraints. Without these queries, stale injection at release 0.5.0 is 36.1% for KANERVA and 31.2% lexically, and the lead over hybrid stays at +0.024. We kept the dataset as it was, so that the tests stay comparable.

The class hazard adds nothing we can measure. Removing it on the main test moves recall by +0.000

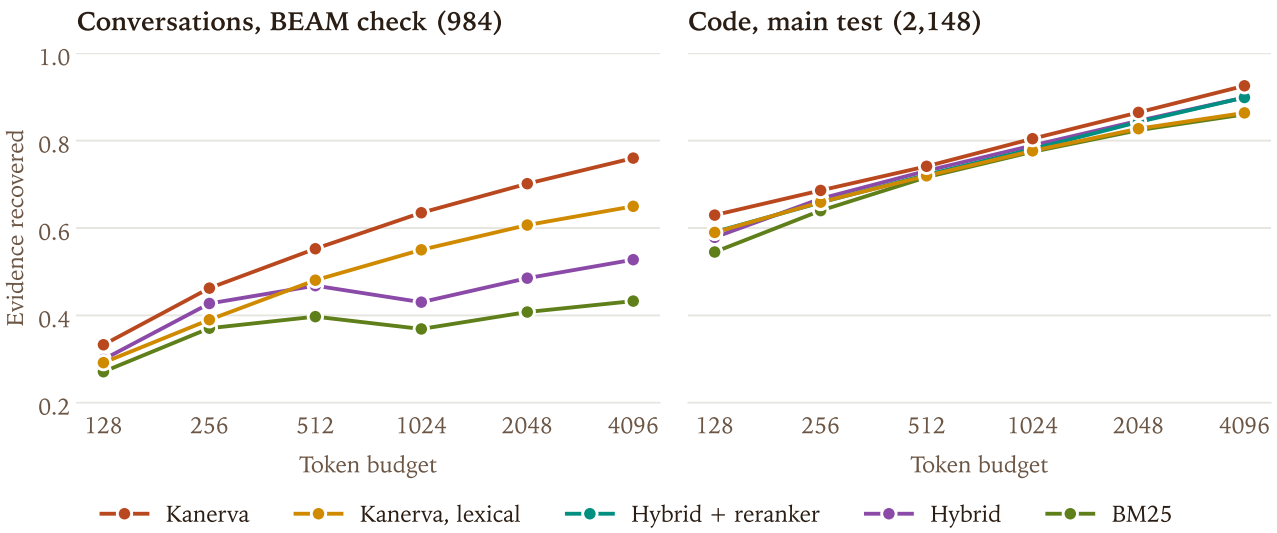


Figure 1: Evidence recall at each token budget. Left: BEAM, on data an earlier test saw. Right: the main test on code, without keys. The reranker was not rerun on BEAM.

(interval [-0.000, +0.000]) and leaves stale injection at 21.0%, so the fall in stale injection comes from version chains and the near-twin rule.

The rule depends on near-identical text. On the *natural-language* updates of LongMemEval, evidence for a fact and for its update are not near-twins, so the rule does not fire, and on development data KANERVA injects the outdated turn on 1.000 of questions against 0.857 for hybrid, since it finds more evidence of both kinds. Listing items newest first keeps the error to 0.029 against 0.471 for hybrid in rank order. Recognising that a free-text statement replaces an older one takes semantics the store does not have, and that is the job of the write path (Section 6.9).

6.6 Long conversations

BEAM-100K and BEAM-500K were never fitted on, but an earlier test saw them, so these results are a check rather than a test. Averaged over budgets KANERVA has 0.574 against 0.440 for hybrid, a difference of +0.134 (interval [+0.124, +0.147], 55/55 conversations in its favour), and at 1024 tokens 0.635 against 0.430. The lexical model has 0.495 against 0.375 for BM25. In the earlier test the reranker had 0.478, and dropping either the wide pool with neighbours or cost density lost +0.016 of recall, a small part of the lead, and we did not isolate the rest. Fitting chats and code at equal weight moved LoCoMo out of fold from 0.733 to 0.726.

6.7 Query gate and real prompts

Without a gate a store with vectors speaks on nearly every prompt. To measure the gate, each store answers its own questions, questions from other stores of the same kind, and questions from another kind of data, at the plugin budget of 600 tokens (Table 8). On the main test's streams with vectors, the share of foreign same-kind questions that get any notes falls from 1.000 to 0.203, and of other-kind questions from 1.000 to 0.000. The cost is on a store's own questions, whose recall falls from 0.657 to 0.620, a relative loss of 5.6%. A lexical store loses 3.0% and still speaks on two in five questions from another repository. On the third C and C++ test with vectors, foreign same-kind questions fall from 0.977 to 0.187.

Those false-alarm figures come from template questions that name identifiers the store has never seen, which are easier to reject than a vague prompt about the store's own project. To see how much that matters we replayed one developer's Claude Code transcripts through the hooks, project by project and in time order, into throwaway stores: 603 prompts in 29 sessions over 5 projects, with prompts from another project standing in for prompts a store cannot answer. Prompts stayed on the machine and the results hold counts only. The gate separates much less here. With vectors, own prompts get notes 53.6% of the time and prompts from other projects 27.7%, and English prompts get notes 69.4% of the time against 37.5% for Russian ones, because these notes are mostly in English. On prompts where a note names a file the turn then touched, the notes a prompt brings name that file only 13.2% of the time, so a real prompt rarely finds the note about the file the agent will open, which motivates file notes.

6.8 File notes

RepoMem's task track gives each query the files of a later commit, so it can simulate two sessions at the two extremes. In the first the agent has just opened one file of the change, and in the second it has just worked on up to three files of the commit before, so its files belong to the previous task. We compare at equal tokens the prompt at 512 tokens plus file notes, against the prompt with those tokens added to its budget, on release 0.5.2 (Table 9).

On the main test with the file open, recall rises from 0.430 to 0.560, against 0.463 at equal tokens, a gain of +0.096, better on 15/15 repositories, for 80 tokens of notes. When the files belong to the previous task recall is 0.445 against 0.478, a loss of -0.032. On the third C and C++ test the two figures are +0.077 and -0.012, with 5/6 repositories up and 0/6 down. Adding the session's files to the prompt query instead hurts exactly when the session has moved on, with lexical recall of 0.338 against 0.399 for the prompt alone, so we dropped it. Task relevance here comes from file overlap, so the first case is generous, and a real session sits somewhere between the two.

6.9 The write path without keys

A note without a key replaces an older one only if someone says so, so `memory_remember` lists similar notes and the host agent decides. We first replay streams with no keys and ask for this list as each note is written, with no LLM involved (Table 10). On the main test with vectors the older version of a changed fact comes first on 0.931 of changed notes and is on the list on 0.994, while natural language is much harder, at 0.511 and 0.809 on LongMemEval updates. The list also appears on 100.0% of notes that replace nothing, so an agent that retired whatever came first would lose good notes, and the value of the list depends on the agent's judgement. An oracle that marks the true older version whenever the list shows it would take stale injection on the main test from 21.0% to 0.3%, which is what a perfect agent could reach.

To measure a real agent, Claude Code with the plugin and Sonnet 5.5 gets a store replayed up to a note and an instruction to remember it, and the store afterwards shows what it did. A *verbatim* note keeps a text word for word. An *own* note is what an agent normally writes: a first session turns the older conversation turn into a short note in its own words, and a second, fresh session does the same with the newer turn. Earlier rounds shaped the hint, the server's instructions and the list, and the sample below is the one they had not seen.

Over 80 trials the agent fixed 10 of 12 changed facts in its own conversation notes, 10 of 12 in word-for-word turns and 16 of 16 on code facts, and it left the store alone in 38 of 40 trials on notes that replace nothing. In three of the four conversation misses the agent gave a defensible reason, such as a list of dated entries that stay true. In 7 trials it changed a note the labels did not mark, and we read them all: most were right or harmless, and one was a real loss, a signature note retired along with a moved function.

6.10 Learning from verdicts

Implicit signals. A natural signal counts a note as used when its identifiers appear in later tool calls. In the transcripts of Section 6.7 the sessions never had these notes, so every firing of this signal is wrong, and it fires on 20.5% of the notes a prompt brought and on 13.0% of random notes from the same store. Good labels alone do not help the benchmarks either. With feedback simulated from evidence labels (Table 11), a linear bandit over shared features loses to the fitted model, at 0.668 against 0.711 on LoCoMo and 0.885 against 0.902 on RepoMem, and the system does not use one. The pooled lift gains little, at 0.727 on LoCoMo and 0.903 on RepoMem. Those benchmarks rarely repeat a question, whereas in the same transcripts 36% of turns that touch files return to a file from an earlier session.

Can an agent tell what helped? Self-reports are known to inflate rewards, so we tested them. Claude Code with Sonnet 5.5 ran 60 sessions on 5 repositories that no earlier test had read. Each session is a small documentation chore that needs two facts, in a store of true notes about that repository, and in some sessions one fact is held only in an older version while the code shows the current one. A shown note is labelled by how the trial was built, as needed, stale or other, and the agent gives its verdict at the stop, as in normal use. Before the first run we required a verdict in at least four of five sessions, at least four in five marks right, at least three in five needed notes marked, at most one in ten other notes marked, and at least half of stale notes called out. All five bars were met (Table 12). The agent judged 100% of sessions, marked 0 of 318 other notes as helpful, and called 10 of 11 stale notes wrong. Those other notes were unrelated facts, which are easy to reject, so these rates are a best case.

Does acting on verdicts help? Not measurably on the build that ships, and no harm was measured either. Six repositories that nothing had read each carry a store of 160 notes, about the size real projects reach, and a chain of 24 sessions. Each session asks for one fact, and a need is drawn to repeat an earlier one with probability 35%, close to the share above. A prompt that names a symbol leaves nothing to learn, since search puts its note first, so a cheap model writes three plain wordings of each need, with no identifier or file name,

used in turn. Both arms get the same store and the same chores in the same order, and both give verdicts at each stop. One arm only records, and the other acts as `KANERVA_LEARN=auto` does. The second half of each chain is scored. An answer is right if it names the thing asked for, or for a constant holds its value. Settings were chosen on spent repositories. Before the run we required a rise in sessions a note helped or in needed notes shown, or a fall in lookups, with an interval clear of zero, while right answers fell by no more than a twentieth and tokens rose by no more than a quarter.

Table 13 gives the results. The limits on the means held and the gain did not appear, so the bar was not met. Right answers were 57 of 72 against 59, with an interval that reaches past the limit, a note helped in 52 sessions in both arms, and lookups were level. The needed note reached the context in 0.472 of sessions against 0.417, and tokens fell from 317.9 to 266.8 per session, both within noise. Lift ran in 2 of the six acting chains and only 3 scored sessions, and the gate moved by itself in 2. The agent judged 133 of 144 sessions in the acting arm and 137 in the other, so counts of sessions a note helped are a little low in both.

Two earlier runs, on other repositories, led to this design. In the first, a build with lift applied to every query met the bar, helping in 71 of 72 sessions against 68, at 81 more tokens per session. In the second, on a later build, right answers fell from 55 to 48 under a score that wanted a signature's whole parameter list word for word. Rerunning two of those repositories with answers kept, the same score gave 15 against 10 of 24, while both arms named the right thing in 20. In that rerun the whole gap lay in the score, which suggests the same of the original fall but does not show it, since the run's own answers were not kept. Word-for-word scores remain in Table 13.

What the gate hides. The chains exposed a larger problem than the one lift addresses. Asked plain questions built the same way, with no agent and with probes off, on new stores from those repositories and the others used in this section, the shipped gate and weight floor bring the needed note to the context on 127 of 480 questions, using 31 tokens of a budget of 600, although it is among the candidates on 420. Moving the gate down by three units gives 235 at 85 tokens, and lowering the floor as well gives 329 at 397. Probes exist to find this project by project, and the gate has begun to move in the chains, but that happened too rarely and too late in a chain to measure what it brings. Replaying the real prompts of Section 6.7, with verdicts stood in for by a weak file-overlap label, no project's gate had moved by the end, and probes, which in this replay opened the gate but did not lower the floor, took tokens per prompt from 184.4 to 189.6.

6.11 Answers from a local model

Recall says what reaches the context and not what a model does with it, so here a small local model answers each question from the notes a method selected. The memory layer still calls no LLM. The reader is Qwen3-4B in 4-bit form on MLX, pinned by revision, told to answer from the notes alone in as few words as it can, or to say unknown. We take 60 questions from LoCoMo, 60 from LongMemEval-S, 40 from BEAM-1M and 90 fact queries from the six Python repositories of the main test, without keys, at 256, 512 and 1024 tokens, on release 0.5.0. Only the last set is unseen. Only short extractive answers can be scored without a judge, and an answer counts as right if it has every word of the reference in any order, or for code the whole value as an unbroken run. Scoring rules were fixed on small pilots before the full run. With no notes the score is zero on every set, and the last row of Table 14 hands the reader the labelled evidence alone.

Across every comparison and metric, 7 of 264 tests reach raw $p < 0.05$ and 0 survive Holm correction, so this test shows no reliable difference between methods. Of the 20 means in Table 15, one reaches raw $p < 0.05$: on the main test's facts Kanerva beats BM25 by +8.5 points (interval +4.5 to +14.4), on all six repositories, which is about what chance gives in that many comparisons. The intervals are wide, so these nulls do not show that methods are equal, but several differences go against Kanerva, and on conversations recall barely converts. At 512 tokens Kanerva leads BM25 in recall by +13.0, +21.5 and +20.5 points on LoCoMo, LongMemEval-S and BEAM-1M, and its answers move by -1.7, +0.0 and -5.0 points. On the main test's facts the same figures are +1.1 and +11.1. We did not test why.

Stale notes do less harm than retrieval figures suggest (Table 16). On 35 sampled facts whose value changed, hybrid and BM25 put a stale note in the context nearly every time, yet hybrid's answers give the old value only 14.3% of the time at 1024 tokens, against 5.7% for Kanerva. The gate saves tokens rather than mistakes: on 160 questions no store can answer, Kanerva invents an answer 4 times without the gate and 4 times with it, because the reader already refuses.

6.12 Russian notes

Russian support adds a Snowball stemmer, stopwords and query patterns, and `KANERVA_EMBEDDER=multilingual` adds search across languages. English behaviour is unchanged. We test on XQuAD [4], which gives SQuAD paragraphs and questions in English and in professional Russian translation, with 1,190 questions over 48 articles. A retrieval counts if it returns the question's paragraph. With a store per article, stemming moves lexical recall

from 0.897 to 0.954 at 1024 tokens (difference +0.056, interval [+0.040, +0.073]). For a Russian question over English notes, lexical search finds almost nothing, at 0.152 in a pooled store, MiniLM fails there too at 0.182, and the multilingual model reaches 0.810. It gains nothing on English and costs more, 127 MB against 91 MB on disk, so MiniLM stays the default. This is not coding-agent memory, and the relevance model has not seen Russian.

6.13 Optional modules and cost

Consolidation. A gold item counts as found if it or its summary is selected (Table 17). Consolidation is not free. At $\tau = 0.9$ it merges only near-duplicates: it saves 2.1% of tokens on the LongMemEval-S sample and 0.5% on LoCoMo, with recall unchanged. At $\tau = 0.8$ it saves 19.9% and 5.6%, and recall moves from 0.969 to 0.789 and from 0.824 to 0.791. At $\tau = 0.7$ the loss is larger. Since the budget caps injected tokens anyway, a smaller store buys little here, so the default threshold is 0.9.

Forgetting. On RepoMem streams, Table 18 compares keeping everything with idle-time rules and token capacities. Forgetting has a real cost on this benchmark. Idle rules of 365 and 180 days cut the active store to 6176 and 3227 tokens from 21492, as Theorem 2 leads one to expect, but recall falls from 0.923 to 0.713 and 0.551, because long-lived facts are read rarely and are still correct. Sessions in this benchmark are 168 days apart, so these idle rules are short in session terms, and the numbers do not transfer directly to a daily workflow. Token capacities behave similarly. Given the latencies below, a store of this size does not need forgetting, so the rules are opt-in. Archived items remain recoverable by hand, but they leave retrieval.

Excerpts. Turn recall credits an excerpt that lost the answer, so we compare on the share of the reference answer's content stems found in the injected text. At 128, 256 and 512 tokens on BEAM, excerpts raise it from 0.280, 0.384 and 0.475 to 0.289, 0.395 and 0.489, and the gain fades by 1024 tokens, so the mode stays off.

Latency. On an M1 Pro, a fresh query at 10^4 items takes 287.1 ms at the median and 359.7 ms at the 95th percentile, plus 8 ms to embed the query, and lexical retrieval takes 150.2 and 210.5 ms. At 10^5 items the medians are 332.4 and 182.2 ms. A hook starts a new process each time, so one prompt at 10^4 items takes 0.72 s in all with MiniLM and 0.49 s lexically.

Live sessions. In real Claude Code sessions on small Python and C++ projects with an isolated store, memory carried over: after one session found that the machine has no `python`, the next went straight to `python3`, and after a C++ session learned that the tests need a Debug build, the next ran that CMake command first. These are anecdotes and not measurements.

Table 2: Main test on code, release 0.5.0: evidence recall. Bold marks the best method per column within a dataset.

Method	128	256	512	1024	2048	4096	mean
RepoMem, keys							
KANERVA + reranker	**0.637**	**0.690**	**0.749**	**0.807**	**0.866**	**0.926**	**0.779**
KANERVA	0.630	0.689	0.744	**0.807**	0.866	**0.926**	0.777
KANERVA, lexical	0.597	0.663	0.724	0.780	0.831	0.867	0.744
KANERVA 0.4.1	0.621	0.676	0.732	0.794	0.859	0.919	0.767
KANERVA 0.4.1, lexical	0.589	0.658	0.721	0.782	0.832	0.865	0.741
Hybrid + reranker	0.591	0.658	0.722	0.781	0.843	0.899	0.749
Hybrid RRF	0.578	0.667	0.731	0.789	0.845	0.899	0.752
Hybrid, live only	0.615	0.677	0.732	0.790	0.846	0.900	0.760
BM25	0.545	0.640	0.716	0.774	0.824	0.860	0.727
RepoMem, no keys							
KANERVA + reranker	**0.630**	**0.688**	**0.746**	**0.804**	**0.865**	0.925	**0.776**
KANERVA	0.630	0.686	0.741	**0.804**	0.865	**0.925**	0.775
KANERVA, lexical	0.590	0.659	0.719	0.777	0.827	0.864	0.739
KANERVA 0.4.1	0.600	0.661	0.718	0.782	0.849	0.909	0.753
KANERVA 0.4.1, lexical	0.570	0.652	0.720	0.782	0.832	0.865	0.737
Hybrid + reranker	0.591	0.658	0.722	0.781	0.843	0.899	0.749
Hybrid RRF	0.579	0.667	0.731	0.789	0.845	0.899	0.752
Hybrid, live only	0.579	0.667	0.731	0.789	0.845	0.899	0.752
BM25	0.545	0.640	0.716	0.774	0.824	0.860	0.727

Table 3: Main test on code: paired differences in recall. Positive favours the first method named. Unit CI resamples repositories, Units + counts those favouring the first method, and p is Holm-corrected across budgets, except on the mean rows. "0.4.1" is an earlier release, and "any twin" drops the query condition on near-twins.

Dataset	Comparison	B	Δ recall	Unit CI	Units +	p
RepoMem, keys	KANERVA vs hybrid	128	+0.051	[+0.023, +0.082]	12/15	0.008
RepoMem, keys	KANERVA vs hybrid	512	+0.013	[+0.006, +0.020]	14/15	0.002
RepoMem, keys	KANERVA vs hybrid	1024	+0.018	[+0.011, +0.025]	12/15	0.008
RepoMem, keys	KANERVA vs hybrid	4096	+0.027	[+0.019, +0.038]	14/15	0.001
RepoMem, keys	KANERVA vs hybrid	mean	+0.025	[+0.016, +0.034]	15/15	<0.001
RepoMem, keys	KANERVA vs reranker	128	+0.039	[+0.024, +0.059]	14/15	<0.001
RepoMem, keys	KANERVA vs reranker	512	+0.022	[+0.014, +0.035]	14/15	<0.001
RepoMem, keys	KANERVA vs reranker	1024	+0.026	[+0.019, +0.037]	15/15	<0.001
RepoMem, keys	KANERVA vs reranker	4096	+0.027	[+0.019, +0.037]	14/15	<0.001
RepoMem, keys	KANERVA vs reranker	mean	+0.028	[+0.021, +0.038]	15/15	<0.001
RepoMem, keys	KANERVA vs hybrid, live	128	+0.014	[+0.005, +0.024]	11/15	0.025
RepoMem, keys	KANERVA vs hybrid, live	512	+0.011	[+0.006, +0.018]	13/15	0.002
RepoMem, keys	KANERVA vs hybrid, live	1024	+0.017	[+0.010, +0.024]	12/15	0.012
RepoMem, keys	KANERVA vs hybrid, live	4096	+0.025	[+0.018, +0.035]	14/15	0.001
RepoMem, keys	KANERVA vs hybrid, live	mean	+0.017	[+0.010, +0.023]	15/15	<0.001
RepoMem, keys	KANERVA vs 0.4.1	128	+0.009	[+0.004, +0.017]	12/15	0.006
RepoMem, keys	KANERVA vs 0.4.1	512	+0.011	[+0.006, +0.020]	12/15	0.006
RepoMem, keys	KANERVA vs 0.4.1	1024	+0.012	[+0.007, +0.021]	14/15	0.001
RepoMem, keys	KANERVA vs 0.4.1	4096	+0.007	[+0.003, +0.014]	12/15	0.006
RepoMem, keys	KANERVA vs 0.4.1	mean	+0.010	[+0.006, +0.015]	14/15	<0.001
RepoMem, keys	KANERVA vs any twin	128	+0.000	[+0.000, +0.000]	0/15	1.000
RepoMem, keys	KANERVA vs any twin	512	+0.000	[+0.000, +0.000]	0/15	1.000
RepoMem, keys	KANERVA vs any twin	1024	+0.000	[+0.000, +0.000]	0/15	1.000
RepoMem, keys	KANERVA vs any twin	4096	+0.000	[+0.000, +0.000]	0/15	1.000
RepoMem, keys	KANERVA vs any twin	mean	+0.000	[+0.000, +0.000]	0/15	1.000
RepoMem, keys	Lexical vs BM25	128	+0.052	[+0.019, +0.089]	11/15	0.063
RepoMem, keys	Lexical vs BM25	512	+0.008	[-0.003, +0.022]	10/15	0.606
RepoMem, keys	Lexical vs BM25	1024	+0.006	[+0.000, +0.011]	11/15	0.453
RepoMem, keys	Lexical vs BM25	4096	+0.007	[+0.002, +0.012]	11/15	0.606
RepoMem, keys	Lexical vs BM25	mean	+0.017	[+0.006, +0.027]	9/15	0.013
RepoMem, keys	Lexical vs 0.4.1	128	+0.009	[+0.002, +0.017]	12/15	0.036
RepoMem, keys	Lexical vs 0.4.1	512	+0.003	[-0.003, +0.007]	10/15	1.000
RepoMem, keys	Lexical vs 0.4.1	1024	-0.002	[-0.005, +0.000]	5/15	0.752
RepoMem, keys	Lexical vs 0.4.1	4096	+0.002	[+0.000, +0.005]	11/15	0.052
RepoMem, keys	Lexical vs 0.4.1	mean	+0.002	[-0.000, +0.005]	11/15	0.492
RepoMem, keys	Lexical vs any twin	128	+0.000	[+0.000, +0.000]	0/15	1.000
RepoMem, keys	Lexical vs any twin	512	+0.000	[+0.000, +0.000]	0/15	1.000
RepoMem, keys	Lexical vs any twin	1024	+0.000	[+0.000, +0.000]	0/15	1.000
RepoMem, keys	Lexical vs any twin	4096	+0.000	[+0.000, +0.000]	0/15	1.000
RepoMem, keys	Lexical vs any twin	mean	+0.000	[+0.000, +0.000]	0/15	1.000
RepoMem, no keys	KANERVA vs hybrid	128	+0.051	[+0.022, +0.082]	12/15	0.009
RepoMem, no keys	KANERVA vs hybrid	512	+0.010	[+0.003, +0.018]	11/15	0.015
RepoMem, no keys	KANERVA vs hybrid	1024	+0.016	[+0.009, +0.023]	12/15	0.015
RepoMem, no keys	KANERVA vs hybrid	4096	+0.027	[+0.018, +0.038]	14/15	0.001
RepoMem, no keys	KANERVA vs hybrid	mean	+0.024	[+0.014, +0.033]	15/15	<0.001
RepoMem, no keys	KANERVA vs reranker	128	+0.039	[+0.023, +0.060]	14/15	<0.001
RepoMem, no keys	KANERVA vs reranker	512	+0.020	[+0.011, +0.035]	13/15	0.001
RepoMem, no keys	KANERVA vs reranker	1024	+0.024	[+0.016, +0.035]	15/15	<0.001
RepoMem, no keys	KANERVA vs reranker	4096	+0.027	[+0.018, +0.038]	14/15	<0.001
RepoMem, no keys	KANERVA vs reranker	mean	+0.026	[+0.018, +0.037]	15/15	<0.001
RepoMem, no keys	KANERVA vs hybrid, live	128	+0.051	[+0.022, +0.081]	12/15	0.009
RepoMem, no keys	KANERVA vs hybrid, live	512	+0.010	[+0.003, +0.019]	11/15	0.015
RepoMem, no keys	KANERVA vs hybrid, live	1024	+0.016	[+0.009, +0.024]	12/15	0.015
RepoMem, no keys	KANERVA vs hybrid, live	4096	+0.027	[+0.018, +0.038]	14/15	0.001
RepoMem, no keys	KANERVA vs hybrid, live	mean	+0.024	[+0.015, +0.033]	15/15	<0.001
RepoMem, no keys	KANERVA vs 0.4.1	128	+0.029	[+0.019, +0.038]	14/15	<0.001
RepoMem, no keys	KANERVA vs 0.4.1	512	+0.023	[+0.016, +0.030]	13/15	<0.001
RepoMem, no keys	KANERVA vs 0.4.1	1024	+0.023	[+0.017, +0.030]	15/15	<0.001
RepoMem, no keys	KANERVA vs 0.4.1	4096	+0.016	[+0.012, +0.021]	14/15	<0.001
RepoMem, no keys	KANERVA vs 0.4.1	mean	+0.022	[+0.017, +0.026]	15/15	<0.001
RepoMem, no keys	KANERVA vs any twin	128	+0.022	[+0.011, +0.030]	6/15	0.188
RepoMem, no keys	KANERVA vs any twin	512	+0.012	[+0.005, +0.018]	6/15	0.188
RepoMem, no keys	KANERVA vs any twin	1024	+0.011	[+0.004, +0.015]	6/15	0.188
RepoMem, no keys	KANERVA vs any twin	4096	+0.008	[+0.003, +0.012]	6/15	0.188
RepoMem, no keys	KANERVA vs any twin	mean	+0.013	[+0.006, +0.018]	6/15	0.031
RepoMem, no keys	Lexical vs BM25	128	+0.045	[+0.013, +0.080]	10/15	0.100
RepoMem, no keys	Lexical vs BM25	512	+0.003	[-0.009, +0.019]	9/15	1.000
RepoMem, no keys	Lexical vs BM25	1024	+0.002	[-0.005, +0.009]	10/15	1.000
RepoMem, no keys	Lexical vs BM25	4096	+0.003	[-0.002, +0.009]	9/15	1.000
RepoMem, no keys	Lexical vs BM25	mean	+0.013	[+0.002, +0.022]	9/15	0.032
RepoMem, no keys	Lexical vs 0.4.1	128	+0.020	[+0.007, +0.033]	13/15	0.008
RepoMem, no keys	Lexical vs 0.4.1	512	-0.000	[-0.005, +0.005]	9/15	1.000
RepoMem, no keys	Lexical vs 0.4.1	1024	-0.005	[-0.007, -0.003]	3/15	0.125
RepoMem, no keys	Lexical vs 0.4.1	4096	-0.002	[-0.004, +0.002]	9/15	1.000
RepoMem, no keys	Lexical vs 0.4.1	mean	+0.003	[-0.000, +0.005]	10/15	0.444
RepoMem, no keys	Lexical vs any twin	128	+0.031	[+0.014, +0.044]	6/15	0.188
RepoMem, no keys	Lexical vs any twin	512	+0.033	[+0.014, +0.047]	6/15	0.188
RepoMem, no keys	Lexical vs any twin	1024	+0.033	[+0.015, +0.047]	6/15	0.188
RepoMem, no keys	Lexical vs any twin	4096	+0.033	[+0.014, +0.048]	6/15	0.188
RepoMem, no keys	Lexical vs any twin	mean	+0.032	[+0.015, +0.047]	6/15	0.031

Table 4: Main test without keys: mean recall over budgets, by language and track. Facts exist only on Python.

Language	Track	KANERVA	Hybrid	Reranker	Lexical	BM25	0.4.1
Python	fact	**0.989**	0.977	0.985	0.969	0.955	0.968
	task	**0.514**	0.444	0.440	0.435	0.413	0.488
TypeScript	task	**0.545**	0.529	0.481	0.521	0.520	0.518
Go	task	**0.523**	0.509	0.486	0.470	0.467	0.498
Rust	task	**0.441**	0.425	0.416	0.396	0.393	0.425

Table 5: Three held-out tests on C and C++, without keys. Differences are in recall, averaged over budgets. Stale injection is at $B = 1024$ on facts that changed. The third test reads more kinds of fact, so it does not compare directly with the first two.

	First	Second	Third
Release	0.5.0	0.5.1	0.5.2
Repositories	redis, curl, jq fmt, spdlog leveldb	nginx, libgit2 libevent, Catch2 benchmark, imgui	lz4, zstd, expat cpp-httplib, CLI11 yaml-cpp
Queries	1,534	1,537	1,461
Over hybrid	+0.014	+0.011	+0.005
interval	[+0.010, +0.017]	[+0.006, +0.016]	[-0.014, +0.023]
repositories	6/6	6/6	4/6
Over reranker	+0.006	+0.012	-0.000
Lexical over BM25	+0.001	+0.009	+0.001
With keys, over hybrid	+0.017	—	+0.023
Stale, KANERVA	51.4%	16.0%	16.4%
Stale, lexical	23.5%	25.5%	20.5%
Stale, hybrid	99.1%	96.9%	98.0%

Table 6: Third C and C++ test without keys: evidence recall on six repositories.

Method	128	256	512	1024	2048	4096	mean
KANERVA	0.806	0.838	0.861	0.885	0.911	0.934	0.873
KANERVA + reranker	0.806	0.838	0.863	0.886	0.912	0.935	0.873
KANERVA, lexical	0.784	0.820	0.848	0.871	0.893	0.906	0.854
KANERVA 0.5.0	**0.815**	**0.848**	**0.870**	**0.895**	**0.922**	**0.945**	**0.882**
KANERVA 0.5.0, lexical	0.779	0.814	0.841	0.864	0.886	0.899	0.847
KANERVA 0.4.1	0.778	0.817	0.844	0.870	0.898	0.926	0.855
Hybrid + reranker	0.801	0.844	0.863	0.886	0.910	0.932	0.873
Hybrid RRF	0.757	0.846	0.868	0.889	0.910	0.932	0.867
Hybrid, newest of twins	0.783	0.835	0.855	0.875	0.893	0.911	0.859
BM25	0.721	0.829	0.862	0.884	0.903	0.916	0.852

Table 7: Main test: staleness on queries whose fact changed, at $B = 1024$.

Method	Keys		No keys	
	inject	error	inject	error
KANERVA	0.000	0.000	0.349	0.003
KANERVA, lexical	0.000	0.000	0.266	0.005
KANERVA 0.4.1	0.000	0.000	0.352	0.003
KANERVA 0.4.1, lexical	0.000	0.000	0.955	0.007
Hybrid, newest of twins	0.365	0.000	0.365	0.003
Hybrid, live only	0.000	0.000	0.999	0.722
Hybrid RRF	0.999	0.723	0.999	0.722
Hybrid + reranker	0.999	0.596	0.999	0.596
BM25	0.990	0.834	0.990	0.834

Table 8: Query gate at 600 tokens. "Spoke" is the share of questions that got any notes. Own questions: their count, recall without and with the gate, and the share still answered with it. Foreign questions: the share that got notes without and with the gate. The main test uses twelve sampled streams and the shipped gates, and other rows are out of fold. RepoMem dev, holdout and earlier tests are repositories used in development.

		Own questions			Same kind, foreign		Other kind, foreign	
Set	n	recall	with gate	spoke	without	with gate	without	with gate
With vectors								
Main test	360	0.657	0.620	0.897	1.000	0.203	1.000	0.000
LongMemEval-S	12	0.979	0.896	0.917	1.000	0.440	0.957	0.137
LoCoMo	300	0.712	0.702	0.977	0.948	0.120	0.816	0.000
BEAM-1M	213	0.457	0.457	0.995	0.997	0.340	0.993	0.057
RepoMem dev	360	0.892	0.845	0.914	0.967	0.307	1.000	0.000
RepoMem holdout	360	0.853	0.846	0.964	0.987	0.423	1.000	0.000
RepoMem, earlier test 1	360	0.844	0.827	0.964	0.987	0.317	0.997	0.000
RepoMem, earlier test 2	360	0.852	0.833	0.950	0.983	0.357	1.000	0.000
RepoMem beyond Python	360	0.531	0.446	0.817	1.000	0.330	1.000	0.000
Lexical								
Main test	360	0.591	0.573	0.942	0.963	0.413	0.617	0.017
LongMemEval-S	12	0.833	0.833	1.000	0.987	0.900	0.987	0.783
LoCoMo	300	0.602	0.602	0.990	0.940	0.392	0.796	0.032
BEAM-1M	213	0.357	0.357	0.977	0.940	0.887	0.850	0.540
RepoMem dev	360	0.839	0.809	0.928	0.993	0.467	0.567	0.013
RepoMem holdout	360	0.816	0.806	0.961	1.000	0.540	0.637	0.043
RepoMem, earlier test 1	360	0.829	0.787	0.925	0.960	0.367	0.510	0.003
RepoMem, earlier test 2	360	0.816	0.788	0.950	0.987	0.517	0.693	0.040
RepoMem beyond Python	360	0.457	0.411	0.883	0.973	0.477	0.587	0.013

Table 9: File notes with vectors. Recall on the task track from the prompt alone, from the prompt with the same tokens added, and from the prompt plus file notes. Precision is that of the notes, and the last columns count repositories where notes beat equal tokens or lose to them.

Set	Session	Prompt	Same tokens	With notes	Precision	Better	Worse
Main test	this task	0.430	0.463	0.560	0.289	15	0
Main test	last task	0.430	0.478	0.445	0.024	0	15
Beyond Python	this task	0.457	0.480	0.555	0.272	9	0
Beyond Python	last task	0.453	0.494	0.466	0.021	0	9
Earlier test 2	this task	0.432	0.466	0.520	0.242	7	1
Earlier test 2	last task	0.427	0.482	0.440	0.020	0	8
Earlier test 1	this task	0.366	0.389	0.423	0.261	4	1
Earlier test 1	last task	0.365	0.403	0.379	0.055	1	4
Holdout	this task	0.451	0.477	0.531	0.250	4	0
Holdout	last task	0.452	0.500	0.463	0.028	0	4
RepoMem dev	this task	0.467	0.497	0.603	0.282	8	0
RepoMem dev	last task	0.453	0.499	0.481	0.037	2	6

Table 10: The write path without keys. Changed notes, how often the older version is first or anywhere on the list of up to five, how often a list appears on notes that replace nothing, stale injection at $B = 1024$ now and with a simulated oracle agent, and the tokens of a list.

Set	Changed	First	Listed	List on others	Stale	Stale, oracle	Tokens
With vectors							
RepoMem, main test	1,904	0.931	0.994	1.000	0.210	0.003	154
RepoMem, earlier test 2	5,279	0.939	0.998	1.000	0.183	0.003	152
RepoMem, earlier test 1	841	0.910	0.993	0.999	0.181	0.007	137
LongMemEval updates	47	0.511	0.809	0.926	1.000	0.319	200
Lexical							
RepoMem, main test	1,904	0.898	0.952	0.530	0.290	0.035	83
RepoMem, earlier test 2	5,279	0.930	0.984	0.612	0.264	0.031	92
RepoMem, earlier test 1	841	0.895	0.975	0.482	0.268	0.028	97
LongMemEval updates	47	0.511	0.723	0.373	0.979	0.340	102

Table 11: Prequential recall at $B = 512$, mean ± s.d. over five seeds, on simulated feedback. "Noisy" flips 20% of rewards.

Dataset	Static	Bandit	Bandit, noisy	Lift	Lift, noisy
LongMemEval-S	0.906 ± 0.000	0.906 ± 0.000	0.906 ± 0.000	0.906 ± 0.000	0.906 ± 0.000
LoCoMo	0.711 ± 0.001	0.668 ± 0.011	0.683 ± 0.010	0.727 ± 0.002	0.706 ± 0.005
RepoMem	0.902 ± 0.002	0.885 ± 0.006	0.895 ± 0.004	0.903 ± 0.002	0.896 ± 0.001

Table 12: Agent verdicts against the implicit signal, on the same 60 sessions.

	Verdict	Implicit
Marks that are right	96.7%	29.2%
interval	89–99%	24–35%
Needed notes marked	78.7%	98.7%
Other notes marked	0.0%	50.9%
Stale notes marked as helpful	1/11	11/11

Table 13: Chains of sessions on six unseen repositories, second half of each chain, mean over repositories, recording only against acting on verdicts. p is a two-sided sign-flip test over repositories, whose smallest possible value here is 0.031. A signature with no parameters counts as word for word.

	Record	Act	Δ	Interval	Up/down	p
Sessions a note helped	0.722	0.722	+0.000	[-0.194, +0.194]	4/2	1.00
Needed note shown	0.417	0.472	+0.056	[-0.014, +0.139]	3/1	0.50
Tokens injected	317.9	266.8	-51.1	[-125.3, +15.1]	1/5	0.22
Lookups before writing	4.82	4.82	+0.00	[-0.42, +0.62]	1/5	1.00
Right answers	0.819	0.792	-0.028	[-0.097, +0.028]	1/2	0.75
Word for word	0.681	0.667	-0.014	[-0.111, +0.097]	1/2	1.00

Table 14: Share of answers that contain the reference, from the notes a method selected, at 256, 512 and 1024 tokens. "Labelled evidence" hands the reader the gold notes alone. With no notes the score is zero.

Method	LoCoMo			LongMemEval-S			BEAM-1M			RepoMem, main test		
	256	512	1024	256	512	1024	256	512	1024	256	512	1024
Kanerva	0.267	0.333	0.350	0.467	0.483	0.483	0.275	0.300	0.375	0.911	0.956	0.900
Kanerva, lexical	0.250	0.283	0.350	0.433	0.500	0.467	0.275	0.350	0.400	0.833	0.889	0.867
Kanerva 0.4.1	0.300	0.317	0.333	0.500	0.550	0.433	0.250	0.400	0.450	0.856	0.878	0.900
Kanerva 0.4.1, lexical	0.233	0.333	0.350	0.467	0.500	0.483	0.300	0.325	0.325	0.833	0.856	0.844
Hybrid RRF	0.233	0.233	0.283	0.367	0.417	0.450	0.350	0.400	0.325	0.867	0.856	0.856
BM25	0.283	0.350	0.350	0.517	0.483	0.450	0.375	0.350	0.350	0.856	0.844	0.811
Labelled evidence		0.467			0.683			0.375			0.922	

Table 15: Answers: paired differences of the mean over 256, 512 and 1024 tokens. The unit is a conversation or a repository, and a question on LongMemEval-S, so its positive-unit count is omitted. p is raw.

Dataset	Comparison	Δ	Interval	Units +	p
LoCoMo	Kanerva vs hybrid	+0.067	[-0.017, +0.144]	7/10	0.199
LoCoMo	Kanerva vs BM25	-0.011	[-0.078, +0.044]	5/10	0.875
LoCoMo	Lexical vs BM25	-0.033	[-0.089, +0.011]	3/10	0.375
LoCoMo	Kanerva vs 0.4.1	+0.000	[-0.028, +0.028]	3/10	1.000
LoCoMo	Lexical vs 0.4.1	-0.011	[-0.044, +0.017]	3/10	0.750
LongMemEval-S	Kanerva vs hybrid	+0.067	[+0.000, +0.139]	—	0.103
LongMemEval-S	Kanerva vs BM25	-0.006	[-0.078, +0.067]	—	1.000
LongMemEval-S	Lexical vs BM25	-0.017	[-0.072, +0.039]	—	0.716
LongMemEval-S	Kanerva vs 0.4.1	-0.017	[-0.061, +0.022]	—	0.612
LongMemEval-S	Lexical vs 0.4.1	-0.017	[-0.044, +0.011]	—	0.453
BEAM-1M	Kanerva vs hybrid	-0.042	[-0.128, +0.037]	4/35	0.427
BEAM-1M	Kanerva vs BM25	-0.042	[-0.111, +0.035]	4/35	0.400
BEAM-1M	Lexical vs BM25	-0.017	[-0.075, +0.044]	3/35	1.000
BEAM-1M	Kanerva vs 0.4.1	-0.050	[-0.117, +0.000]	1/35	0.186
BEAM-1M	Lexical vs 0.4.1	+0.025	[-0.017, +0.067]	5/35	0.380
RepoMem, main test	Kanerva vs hybrid	+0.063	[+0.007, +0.154]	4/6	0.125
RepoMem, main test	Kanerva vs BM25	+0.085	[+0.045, +0.144]	6/6	0.031
RepoMem, main test	Lexical vs BM25	+0.026	[-0.051, +0.100]	4/6	0.375
RepoMem, main test	Kanerva vs 0.4.1	+0.044	[-0.008, +0.095]	5/6	0.156
RepoMem, main test	Lexical vs 0.4.1	+0.019	[-0.051, +0.086]	3/6	0.781

Table 16: Answers on facts that changed, main test: percent of answers that give the old value, and of queries with a stale note in the context.

Method	Old value answered			Stale note injected		
	256	512	1024	256	512	1024
Kanerva	0.0	0.0	5.7	37.1	40.0	40.0
Kanerva, lexical	0.0	2.9	5.7	45.7	45.7	45.7
Kanerva 0.4.1	2.9	5.7	2.9	40.0	42.9	42.9
Kanerva 0.4.1, lexical	25.7	17.1	17.1	94.3	94.3	97.1
Hybrid RRF	8.6	11.4	14.3	97.1	100.0	100.0
BM25	14.3	14.3	20.0	97.1	97.1	100.0

Table 17: Reversible consolidation (LongMemEval-S: 60 random questions). τ is the similarity threshold, R the extractive rate.

Dataset, τ, rate	Clusters	Sources	Tokens saved	Recall@1024
LoCoMo, baseline	0	0	0.0%	0.824
LoCoMo, $\tau = 0.7$, $R = 0.5$	577	1972	23.0%	0.669
LoCoMo, $\tau = 0.8$, $R = 0.5$	195	535	5.6%	0.791
LoCoMo, $\tau = 0.9$, $R = 0.5$	29	71	0.5%	0.823
LongMemEval-S, baseline	0	0	0.0%	0.969
LongMemEval-S, $\tau = 0.7$, $R = 0.5$	7985	20806	39.4%	0.689
LongMemEval-S, $\tau = 0.8$, $R = 0.5$	4625	10327	19.9%	0.789
LongMemEval-S, $\tau = 0.9$, $R = 0.5$	458	1015	2.1%	0.969

Table 18: Forgetting on RepoMem streams: final active store and recall.

Policy	Active items	Active tokens	Recall@1024
keep-all	965	21492	0.923
idle-730d	474	11661	0.857
idle-365d	243	6176	0.713
idle-180d	130	3227	0.551
cap-8k	426	7986	0.750
cap-4k	239	3989	0.540

7 Limitations

Outcomes. Most metrics are retrieval-level. The answer test rests on one small reader, short extractive questions and small samples, so its nulls do not show that methods are equal, and a stronger model may use notes differently. The chains measure a real agent on small documentation chores, not on real tasks, and their answers are scored by string match on a name or value. How much recall per token turns into finished work is still unknown.

Baselines. Mem0 with inference, Graphiti, A-MEM, HippoRAG, LangMem and Cognee all call an LLM on write, so the comparison does not cover them, and it says nothing about what LLM extraction would add to those systems. Published evidence suggests a small effect on the write side [68].

Generalisation. The relevance model is fitted on four benchmark families, with BEAM-1M the only source of long conversations, so the BEAM results concern new conversations from a family the model has seen, and no untouched conversation benchmark is left. LongMemEval-S and LoCoMo informed the design and are not clean. Exact code facts are read only for Python, C and C++, so nothing here says how they behave in TypeScript, Go or Rust, and each of those languages rests on three repositories. A deployment on notes unlike these may behave differently. Beyond English, only Russian was measured, on XQuAD and not on coding notes.

RepoMem is a proxy. Queries come from templates over extracted facts, so they contain identifiers and favour lexical search, while real prompts are vaguer, and on plain-language questions the gate and the weight floor hide most notes (Section 6.10). The agent's notes are simulated from code and commit history, and every changed fact is assumed to be observed. The task track uses file overlap as relevance, which is a weak label. Keyed mode gets perfect slot keys from the static analyser, so it is an upper bound on what a real write path provides, and plain mode is the conservative end.

Validity. The near-twin rule works where versions differ only in a value. It does not fire on natural-language updates, and without keys it still lets an outdated version through 21.0% of the time on the main test. Those updates depend on a host agent, which was measured on one model and a few dozen notes, and which sometimes retired more than it should. Without vectors the list misses most paraphrases, and it appears on many notes that replace nothing, each time costing the agent tokens.

The gate. It costs recall on a store's own questions, and those queries get nothing when it closes. Its false-alarm rates come from foreign questions, and on one developer's real prompts it separates far less. How often it is wrongly silent in real use is not known.

Learning. A verdict covers only notes the agent was shown, so the shadow check cannot see what either ordering left out. Verdicts are tied to a project rather than a session, so two sessions open on one project can mix. The signal test labels notes by construction, and its distractors are easy to reject. The chains use one model, small chores and six repositories at a time. A project's gate moved in only two of them, late in the chain, so per-project tuning over longer use is unmeasured.

Plugins. The benchmarks ran the library with exact token counts, while the plugins estimate tokens with a dependency-free approximation. Until the embedding model arrives, a new install runs the lexical model, so its first sessions get the lexical numbers.

8 Conclusion

Treating agent memory as budgeted selection gives a read path that needs no LLM, and on code its strongest lever turned out to be simple: a calibrated estimate of what each candidate is worth. On fifteen repositories in four languages it beats hybrid retrieval and a cross-encoder by a few points of recall, with or without keys, and three tests on C and C++ show a smaller lead that fails to hold on the third once class methods count as facts. On long conversations the lead is far larger. Stale injection is a serious problem on code facts, and version chains and simple near-twin rules remove most of it, while free-text updates need a host agent, which fixes most of them when shown similar notes. A small local model answering from the selected notes showed no reliable difference between methods, so the recall gains are not yet shown to reach answers. A real agent's verdicts turned out to be a reliable signal where matching identifiers in tool calls was not, and learning from them on chains of sessions has shown no measurable gain or harm. The largest open problem is silence on vague prompts, where the note is often in the store and the gate and the floor keep it out.

The next step is to make the gate speak on vague prompts without flooding unrelated ones, and to measure end to end whether any of this changes what a coding agent gets done.

Reproducibility. `make data && make reproduce` rebuilds the benchmark results, the tests with a real agent are run by hand with the scripts named in

`experiments/`, `make check` verifies each number in this paper against `results/*.json`, and the RepoMem manifest pins every repository snapshot.

References

[1] John R. Anderson and Lael J. Schooler. Reflections of the Environment in Memory. *Psychological Science*, 2:396–408, 1991. doi: 10.1111/j.1467-9280.1991.tb00174.x. URL https://doi.org/10.1111/j.1467-9280.1991.tb00174.x.

[2] John R. Anderson, Daniel Bothell, Michael D. Byrne, Scott Douglass, Christian Lebiere, and Yulin Qin. An Integrated Theory of the Mind. *Psychological Review*, 111:1036–1060, 2004. doi: 10.1037/0033-295X.111.4.1036. URL https://doi.org/10.1037/0033-295X.111.4.1036.

[3] Anthropic. How Claude remembers your project, 2026. URL https://code.claude.com/docs/en/memory. Accessed October 2026.

[4] Mikel Artetxe, Sebastian Ruder, and Dani Yogatama. On the Cross-lingual Transferability of Monolingual Representations. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4623–4637, 2020. doi: 10.18653/v1/2020.acl-main.421. URL https://doi.org/10.18653/v1/2020.acl-main.421.

[5] Mohammad Asadolahi, Amir Amini, Samira Talebi, Amirfarhad Farhadi, and Azadeh Zamanifar. Memory Reward Inflation in Self-Improving LLM Agents, 2026. URL https://arxiv.org/abs/2608.00017.

[6] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to Memorize at Test Time, 2024. URL https://arxiv.org/abs/2501.00663.

[7] Vincent-Pierre Berges, Barlas Oğuz, Daniel Haziza, Wen tau Yih, Luke Zettlemoyer, and Gargi Ghosh. Memory Layers at Scale, 2024. URL https://arxiv.org/abs/2412.09764.

[8] Jaime Carbonell and Jade Goldstein. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 335–336, 1998. doi: 10.1145/290941.291025. URL https://doi.org/10.1145/290941.291025.

[9] Wei Chen, Yajun Wang, and Yang Yuan. Combinatorial Multi-Armed Bandit: General Framework and Applications. In *International Conference on Machine Learning*, volume 28 of *PMLR*, pages 151–159, 2013. URL https://proceedings.mlr.press/v28/chen13a.html.

[10] Wei Chen, Yajun Wang, Yang Yuan, and Qinshi Wang. Combinatorial Multi-Armed Bandit and Its Extension to Probabilistically Triggered Arms, 2014. URL https://arxiv.org/abs/1407.8339.

[11] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting Language Models to Compress Contexts, 2023. URL https://arxiv.org/abs/2305.14788.

[12] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory, 2025. URL https://arxiv.org/abs/2504.19413.

[13] Ashwin Gerard Colaco and Nada Lahjouji. What to Keep, What to Forget: A Rate-Distortion View of Memory Compaction in LLMs and Agents, 2026. URL https://arxiv.org/abs/2607.08032.

[14] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009. doi: 10.1145/1571941.1572114. URL https://doi.org/10.1145/1571941.1572114.

[15] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From Local to Global: A Graph RAG Approach to Query-Focused Summarization, 2024. URL https://arxiv.org/abs/2404.16130.

[16] Liyang Fan, Yingcheng Shi, Yongbin Li, Chenghao Sun, Xin Chen, Xander Xu, Hu Wei, Shiwen Ni, Min Yang, and Jieping Ye. VibeMemBench: Evaluating Memory Systems for Coding Agents on Real Repository Coding Tasks, 2026. URL https://arxiv.org/abs/2609.23570.

[17] Moran Feldman, Zeev Nutov, and Elad Shoham. Practical Budgeted Submodular Maximization, 2020. URL https://arxiv.org/abs/2007.04937.

[18] Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. In-context Autoencoder for Context Compression in a Large Language Model, 2023. URL https://arxiv.org/abs/2307.06945.

[19] Manu Ghulyani, Arunabh Singh, Karan Bharadwaj, Ankit Nath, and Suranjan Goswami. PACMS: Submodular Context Selection as a Pluggable Engine for LLM Agents, 2026. URL https://arxiv.org/abs/2606.20047.

[20] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. LightRAG: Simple and Fast Retrieval-Augmented Generation, 2024. URL https://arxiv.org/abs/2410.05779.

[21] Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. From RAG to Memory: Non-Parametric Continual Learning for Large Language Models. In *International Conference on Machine Learning (ICML)*, 2025. URL https://arxiv.org/abs/2502.14802.

[22] Yudai Honda, Keita Fujita, Keiichi Zempo, and Shogo Fukushima. Human-Like Remembering and Forgetting in LLM Agents: An ACT-R-Inspired Memory Architecture. In *International Conference on Human-Agent Interaction*, pages 229–237, 2025. doi: 10.1145/3765766.3765803. URL https://doi.org/10.1145/3765766.3765803.

[23] Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://arxiv.org/abs/2507.05257.

[24] Mehmet Iscan. PYTHALAB-MERA: Validation-Grounded Memory, Retrieval, and Acceptance Control for Frozen-LLM Coding Agents, 2026. URL https://arxiv.org/abs/2605.08468.

[25] Mehmet Iscan. Learning When to Remember: Risk-Sensitive Contextual Bandits for Abstention-Aware Memory Retrieval in LLM-Based Coding Agents, 2026. URL https://arxiv.org/abs/2604.27283.

[26] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models, 2023. URL https://arxiv.org/abs/2310.05736.

[27] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression, 2023. URL https://arxiv.org/abs/2310.06839.

[28] Prakhar Khatri. Do Context Files Help Coding Agents? A Two-Agent Ablation Study on Real Repositories, 2026. URL https://arxiv.org/abs/2607.27250.

[29] Samir Khuller, Anna Moss, and Joseph (Seffi) Naor. The budgeted maximum coverage problem. *Information Processing Letters*, 70:39–45, 1999. doi: 10.1016/S0020-0190(99)00031-9. URL https://doi.org/10.1016/S0020-0190(99)00031-9.

[30] Andreas Krause and Carlos Guestrin. A Note on the Budgeted Maximization of Submodular Functions. Technical Report CMU-CALD-05-103, Carnegie Mellon University, 2005. URL http://reports-archive.adm.cs.cmu.edu/anon/cald/CMU-CALD-05-103.pdf. Accessed October 2026.

[31] Alex Kulesza and Ben Taskar. Determinantal point processes for machine learning, 2012. URL https://arxiv.org/abs/1207.6083.

[32] Dharshan Kumaran, Demis Hassabis, and James L. McClelland. What Learning Systems do Intelligent Agents Need? Complementary Learning Systems Theory Updated. *Trends in Cognitive Sciences*, 20:512–534, 2016. doi: 10.1016/j.tics.2016.05.004. URL https://doi.org/10.1016/j.tics.2016.05.004.

[33] Branislav Kveton, Zheng Wen, Azin Ashkan, and Csaba Szepesvari. Tight Regret Bounds for Stochastic Combinatorial Semi-Bandits, 2014. URL https://arxiv.org/abs/1410.0949.

[34] Jure Leskovec, Andreas Krause, Carlos Guestrin, Christos Faloutsos, Jeanne VanBriesen, and Natalie Glance. Cost-effective outbreak detection in networks. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 420–429, 2007. doi: 10.1145/1281192.1281239. URL https://doi.org/10.1145/1281192.1281239.

[35] Han Li, Letian Zhu, Bohan Zhang, Rili Feng, Jiaming Wang, Yue Pan, Earl T. Barr, Federica Sarro, Zhaoyang Chu, and He Ye. ContextBench: A Benchmark for Context Retrieval in Coding Agents, 2026. URL https://arxiv.org/abs/2602.05892.

[36] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM Knows What You are Looking for Before Generation, 2024. URL https://arxiv.org/abs/2404.14469.

[37] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating Very Long-Term Conversational Memory of LLM Agents. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. URL https://arxiv.org/abs/2402.17753.

[38] James L. McClelland, Bruce L. McNaughton, and Randall C. O'Reilly. Why there are complementary learning systems in the hippocampus and

neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102:419–457, 1995. doi: 10.1037/0033-295X.102.3.419. URL https://doi.org/10.1037/0033-295X.102.3.419.

[39] Michel Minoux. Accelerated greedy algorithms for maximizing submodular set functions. In *Optimization Techniques*, Lecture Notes in Control and Information Sciences, pages 234–243. 1978. doi: 10.1007/BFb0006528. URL https://doi.org/10.1007/BFb0006528.

[40] Kazuki Nakayashiki. When Stale Constraints Go Unchecked: Budgeted Verification Failures in Inherited Agent Memory, 2026. URL https://arxiv.org/abs/2608.25553.

[41] George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. An analysis of approximations for maximizing submodular set functions—I. *Mathematical Programming*, 14:265–294, 1978. doi: 10.1007/BF01588971. URL https://doi.org/10.1007/BF01588971.

[42] Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory, 2025. URL https://arxiv.org/abs/2509.25140.

[43] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as Operating Systems, 2023. URL https://arxiv.org/abs/2310.08560.

[44] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression, 2024. URL https://arxiv.org/abs/2403.12968.

[45] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior, 2023. URL https://arxiv.org/abs/2304.03442.

[46] Chao Peng, Bin Wang, Zhilei Long, and Jinfang Sheng. AdaGReS:Adaptive Greedy Context Selection via Redundancy-Aware Scoring for Token-Budgeted RAG, 2025. URL https://arxiv.org/abs/2512.25052.

[47] Aleksandar Petrov, Mark Sandler, Andrey Zhmoginov, Nolan Miller, and Max Vladymyrov. Long Context In-Context Compression by Getting to the Gist of Gisting, 2025. URL https://arxiv.org/abs/2504.08934.

[48] Nathanaël Carraz Rakotonirina, Mohammed Hamdy, Jon Ander Campos, Lucas Weber, Alberto Testoni, Marzieh Fadaee, Sandro Pezzelle, and Marco Del Tredici. From Tools to Teammates: Evaluating LLMs in Multi-Session Coding Interactions, 2025. URL https://arxiv.org/abs/2502.13791.

[49] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A Temporal Knowledge Graph Architecture for Agent Memory, 2025. URL https://arxiv.org/abs/2501.13956.

[50] Stephen E. Robertson, Steve Walker, Susan Jones, Micheline Hancock-Beaulieu, and Mike Gatford. Okapi at TREC-3. In *Text REtrieval Conference (TREC-3)*, pages 109–126, 1994. URL https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/okapi_trec3.pdf.

[51] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval, 2024. URL https://arxiv.org/abs/2401.18059.

[52] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning, 2023. URL https://arxiv.org/abs/2303.11366.

[53] Baixi Sun, Le Chen, Anjir Ahmed Chowdhury, Xiaolong Ma, Chih-Hsuan Yang, Mingze Xia, Syed Zawad, Sheng Di, Rajkumar Kettimuthu, Huihuo Zheng, Rajeev Thakur, Venkatram Vishwanath, and Feng Yan. LAM: Efficient Lossy Agent Memory Framework With A Retrieval-Score Error Bound, 2026. URL https://arxiv.org/abs/2609.32256.

[54] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-Time Training with Self-Supervision for Generalization under Distribution Shifts, 2019. URL https://arxiv.org/abs/1909.13231.

[55] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to

(Learn at Test Time): RNNs with Expressive Hidden States, 2024. URL https://arxiv.org/abs/2407.04620.

[56] Maxim Sviridenko. A note on maximizing a submodular set function subject to a knapsack constraint. *Operations Research Letters*, 32:41–43, 2004. doi: 10.1016/S0167-6377(03)00062-2. URL https://doi.org/10.1016/S0167-6377(03)00062-2.

[57] Sho Takemori, Masahiro Sato, Takashi Sonoda, Janmajay Singh, and Tomoko Ohkuma. Submodular Bandit Problem Under Multiple Constraints, 2020. URL https://arxiv.org/abs/2006.00661. PMLR 124 (UAI 2020).

[58] Jing Tang, Xueyan Tang, Andrew Lim, Kai Han, Chongshou Li, and Junsong Yuan. Revisiting Modified Greedy Algorithm for Monotone Submodular Maximization with a Knapsack Constraint, 2020. URL https://arxiv.org/abs/2008.05391.

[59] Zhen Tao, Jinxiang Zhao, Peng Liu, Dinghao Xi, Yanfang Chen, Wei Xu, and Zhiyu Li. MemConflict: Evaluating Long-Term Memory Systems Under Memory Conflicts, 2026. URL https://arxiv.org/abs/2605.20926.

[60] Mohammad Tavakoli, Alireza Salemi, Carrie Ye, Mohamed Abdalla, Hamed Zamani, and J Ross Mitchell. Beyond a Million Tokens: Benchmarking and Enhancing Long-Term Memory in LLMs. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://arxiv.org/abs/2510.27246.

[61] Sentence Transformers. sentence-transformers/all-MiniLM-L6-v2 model card, 2026. URL https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2. Accessed October 2026.

[62] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models, 2023. URL https://arxiv.org/abs/2305.16291.

[63] Zheng Wen, Branislav Kveton, and Azin Ashkan. Efficient Learning in Large-Scale Combinatorial Semi-Bandits, 2014. URL https://arxiv.org/abs/1406.7443.

[64] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2410.10813.

[65] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient Streaming Language Models with Attention Sinks, 2023. URL https://arxiv.org/abs/2309.17453.

[66] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic Memory for LLM Agents, 2025. URL https://arxiv.org/abs/2502.12110.

[67] Baosheng Yu, Meng Fang, and Dacheng Tao. Linear Submodular Bandits with a Knapsack Constraint. In *AAAI Conference on Artificial Intelligence*, volume 30, 2016. doi: 10.1609/aaai.v30i1.10154. URL https://doi.org/10.1609/aaai.v30i1.10154.

[68] Boqin Yuan, Yue Su, and Kun Yao. Diagnosing Retrieval vs. Utilization Bottlenecks in LLM Agent Memory, 2026. URL https://arxiv.org/abs/2603.02473.

[69] Yisong Yue and Carlos Guestrin. Linear Submodular Bandits and their Application to Diversified Retrieval. In *Advances in Neural Information Processing Systems 24*, pages 2483–2491, 2011. URL https://proceedings.neurips.cc/paper/2011/hash/33ebd5b07dc7e407752fe773eed20635-Abstract.html.

[70] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models, 2025. URL https://arxiv.org/abs/2510.04618.

[71] Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Zhuo Li, Yujie Zheng, Weinan Zhang, Ying Wen, Zhiyu Li, Feiyu Xiong, Yutao Qi, Bo Tang, and Muning Wen. MemRL: Self-Evolving Agents via Runtime Reinforcement Learning on Episodic Memory, 2026. URL https://arxiv.org/abs/2601.03192.

[72] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H_2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models, 2023. URL https://arxiv.org/abs/2306.14048.

[73] Ziyi Zhang, Shuang Cui, Haotian Zhang, and Xiaoyu Wang. PC-SubMax: Efficient Prompt Compression via Regularized Submodular Maximization, 2026. URL https://arxiv.org/abs/2609.32474.

[74] Jiayuan Zhu, Junde Wu, Minhao Hu, Shengda Zhu, Jiazhen Pan, Weixiang Shen, Yijun Yang, Fenglin Liu, Jianye Hao, Yueming Jin, Qirong Ho, and Min Xu. SWE Context Bench: A Benchmark for Context Learning in Coding, 2026. URL https://arxiv.org/abs/2602.08316.

[75] Mingxi Zou, Zhihan Guo, Langzhang Liang, Zhuo Wang, Qifan Wang, Qingsong Wen, Irwin King, Lizhen Qu, and Zenglin Xu. Remember the Decision, Not the Description: A Rate-Distortion Framework for Agent Memory, 2026. URL https://arxiv.org/abs/2605.10870.

A Proofs

Proof of Proposition 1. Take additive U. One item a has score $1+\epsilon$ and cost B. There are $\lfloor B/c_{\min} \rfloor$ items of score 1 and cost $c_{\min}$. The rule fills with a first and then has no budget, so it earns $1+\epsilon$ against $\mathrm{OPT} \geq \lfloor B/c_{\min} \rfloor$. Let $\epsilon \to 0$. □

Proof of Proposition 2. Walking from newest to oldest, keep each item that fits. Since the later items cost more than B in total, one of them fails to fit, and at that moment the remaining budget is below its cost, which is at most $c(m)$. The remaining budget only shrinks afterwards, so m never fits. A relevance rule takes m first when it outranks those items, and $c(m) \leq B$ then lets it in. □

Proof of Proposition 3. An invariant score cannot depend on how the two timestamps are assigned, so the ranking between the two versions is independent of which one is current. Under a uniform assignment the outdated version ranks first with probability 1/2. □

Proof of Proposition 4. Sort the items by density w/c and let G be the prefix accepted before the first item g that does not fit. If no item fails to fit, G holds every item of positive weight and is optimal. Otherwise $G \cup \{g\}$ is a density-ordered prefix whose cost exceeds the budget, so its value is at least the optimum of the fractional relaxation, and hence at least OPT. Thus $F(G) + w_g \geq \mathrm{OPT}$, and the larger of the two terms is at least $\frac{1}{2}\mathrm{OPT}$. The density run contains G, so it has at least $F(G)$, and the best single item has at least w_g because $c(g) \leq B$. □

Proof of Proposition 5. For any set S, $|\sum_{m \in S} p_m - \sum_{m \in S} \hat{p}_m| \leq e(S)$. Hence $\sum_{\hat{S}} p \geq \sum_{\hat{S}} \hat{p} - e(\hat{S}) \geq \alpha \sum_{S^\star} \hat{p} - e(\hat{S}) \geq \alpha \sum_{S^\star} p - \alpha e(S^\star) - e(\hat{S})$, where the middle step uses that $S^\star$ is feasible. For $\alpha = 1$, the items in $S^\star \cap \hat{S}$ cancel: apply the same chain to $S^\star \setminus \hat{S}$ and $\hat{S} \setminus S^\star$, using $\sum_{\hat{S} \setminus S^\star} \hat{p} \geq \sum_{S^\star \setminus \hat{S}} \hat{p}$, which holds because $\hat{S}$ is optimal for $\hat{p}$. □

Proof of Proposition 6. Let each round have n items with equal features, $\mu = 1$, $c = 1$, $B = 1$ and $F(S; w) = \sum_{i \in S} w_i$, with the index of the valid item uniform and independent across rounds. What a blind learner sees in past rounds is independent of the current valid index, so whichever item it picks is valid with probability $1/n$ and $\mathbb{E}F_t(S_t) = 1/n$, while $\mathrm{OPT}_t = 1$. A learner that sees the version chain picks the current item in every round. □

Proof of Theorem 1. Φ is monotone submodular (facility location), $\Phi(\emptyset) = 0$ and $\Phi(X) = 1$. Let g_i be the density-maximal pick, with gain $\Delta_i = \Phi(Z_i) - \Phi(Z_{i-1})$. Monotonicity and submodularity give $\Phi(Z^\star) \leq \Phi(Z^\star \cup Z_{i-1}) \leq \Phi(Z_{i-1}) + \sum_{z \in Z^\star \setminus Z_{i-1}} \Delta(z \mid Z_{i-1}) \leq \Phi(Z_{i-1}) + R^\star \Delta_i / c(g_i)$. So $d_i := \Phi(Z^\star) - \Phi(Z_i) \leq (1 - c(g_i)/R^\star) d_{i-1} \leq e^{-c(g_i)/R^\star} d_{i-1}$ whenever $d_{i-1} > 0$, using $1 - x \leq e^{-x}$ for all real x. If $d_{i-1} \leq 0$ for some i, then $\Phi(Z_\ell) \geq \Phi(Z^\star)$ by monotonicity. In every case $\Phi(Z_\ell) \geq (1 - e^{-R_\ell / R^\star}) \Phi(Z^\star)$, and substituting $\Phi = 1 - D$ gives the claim. □

Proof of Theorem 2. Condition on r and let T be the time until the first gap of length τ in a Poisson process of rate r started at the arrival. Let $A \sim \mathrm{Exp}(r)$ be the first access. If $A > \tau$ then $T = \tau$, otherwise $T = A + T'$ with T' an independent copy of T by the memoryless property. So $\mathbb{E}T = \mathbb{E}\min(A, \tau) + (1 - e^{-r\tau})\mathbb{E}T$. With $\mathbb{E}\min(A, \tau) = (1 - e^{-r\tau})/r$, this gives $\mathbb{E}T = (e^{r\tau} - 1)/r$.

The pairs (arrival time, lifetime) form a marked Poisson process with i.i.d. marks. By the marking theorem, the number of items alive at t is Poisson with mean $a \int_0^t \Pr[L > u]\, du$, which converges to $a\,\mathbb{E}L$ under the finiteness assumption. □

B Earlier tests

Three earlier evaluations of older releases shaped the method. Their result files remain in the repository, and their repositories later joined the fit.

Repositories	Finding	Effect on the method
5 Python	Near-twin rule took look-alike commit notes for versions of a fact	Only facts replace each other
8 Python	With keys ahead of every rival, without keys level with hybrid	Twins must cover the query
9 TypeScript, Go, Rust	No lead for a model fitted on Python alone (-0.006, [-0.015, +0.003])	Those repositories joined the fit